Lab 21 – Collaborative AI Coding: Pair Programming and Version Control Integration

Task 1 – AI-Assisted Pair Programming

Task:

- Use AI to generate an initial program (e.g., a simple To-Do List Manager).
- Ask AI to suggest enhancements (such as error handling, additional fieatures, or refiactoring).
- Compare the original and improved versions to understand the role ofi AI in collaboration.

Prompt

> "Act as my pair programmer. First, generate a very basic Python To-Do List manager that stores tasks in memory. Then, suggest enhancements fior error handling and data persistence, and provide the refiactored improved code."

Code

```python
import os
# Original Version (Basic Memory-only)
# tasks = []
# def add_task(t): tasks.append(t)
# def show_tasks(): print(tasks)

# Improved Version (AI-Assisted Enhancements)
class TodoManager:
    def __init__(self, filename="todo_list.txt"):
        self.filename = filename
        self.tasks = self.load_tasks()

    def load_tasks(self):
        """Loads tasks with error handling."""
        if not os.path.exists(self.filename):
            return []
        try:
            with open(self.filename, 'r') as file:
                return [line.strip() for line in file.readlines()]
        except IOError as e:
            print(f"Error loading tasks: {e}")
            return []

    def add_task(self, task):
        """Adds a task and persists it to a file."""
        if not task.strip():
            print("Error: Task cannot be empty.")
            return
        self.tasks.append(task)
        self.save_tasks()
        print(f"Added: '{task}'")

    def save_tasks(self):
        try:
            with open(self.filename, 'w') as file:
                for task in self.tasks:
                    file.write(task + '\n')
        except IOError as e:
            print(f"Error saving tasks: {e}")

# Testing the improved code
todo = TodoManager()
todo.add_task("Complete AI Lab 21")
todo.add_task("Submit Assignment")
```

Output

```
Added: 'Complete AI Lab 21'
Added: 'Submit Assignment'
```

Explanation

The AI acted as an effective pair programmer by first outlining the core logic and then identifiying critical gaps like data loss upon exit. The improved version incorporates file I/O operations and exception handling, demonstrating how AI can rapidly elevate a firagile prototype into a robust application.

Task 2 – Version Control Integration

Task: Initialize a Git repository fior your project. Perfiorm the fiollowing steps:

- Create and switch to a new branch.
- Add AI-assisted modifications.
- Commit with a meaningfiul message.
- Merge changes back into the main branch.
- Reflect on how version control supports collaborative development with AI assistance.

Prompt

> "Provide the exact Git terminal commands required to initialize a local repository, create a new branch called 'fieature-ai-enhancements', stage an updated 'todo.py' file, commit it, and finally merge those changes back into the 'main' branch."

Code (Git Commands)

```
# 1. Initialize repository
git init
git branch -m main

# 2. Create and switch to new branch
git checkout -b feature-ai-enhancements

# 3. Add AI-assisted modifications
git add todo.py

# 4. Commit with a meaningful message
git commit -m "feat: integrate AI-suggested error handling and file storage"

# 5. Merge changes back into main
git checkout main
git merge feature-ai-enhancements
```

Output

```
Initialized empty Git repository in C:/Users/nikhi/Projects/AI Assistant Coding/TestRepo/.git/
PS C:\Users\nikhi\Projects\AI Assistant Coding\TestRepo> git branch -m main
PS C:\Users\nikhi\Projects\AI Assistant Coding\TestRepo> git add todo.py
fatal: pathspec 'todo.py' did not match any files
PS C:\Users\nikhi\Projects\AI Assistant Coding\TestRepo> git add todo.py
PS C:\Users\nikhi\Projects\AI Assistant Coding\TestRepo> git commit -m "feat: integrate AI-suggested error handling and file storage"
[main (root-commit) e4e42d5] feat: integrate AI-suggested error handling and file storage
 1 file changed, 45 insertions(+)
 create mode 100644 todo.py
PS C:\Users\nikhi\Projects\AI Assistant Coding\TestRepo> git checkout main
Already on 'main'
PS C:\Users\nikhi\Projects\AI Assistant Coding\TestRepo> git checkout -b feature-ai-enhancements
Switched to a new branch 'feature-ai-enhancements'
PS C:\Users\nikhi\Projects\AI Assistant Coding\TestRepo> git add todo.py
PS C:\Users\nikhi\Projects\AI Assistant Coding\TestRepo> git commit -m "feat: integrate AI-suggested error handling and file storage"
On branch feature-ai-enhancements
nothing to commit, working tree clean
PS C:\Users\nikhi\Projects\AI Assistant Coding\TestRepo> git checkout main
Switched to branch 'main'
PS C:\Users\nikhi\Projects\AI Assistant Coding\TestRepo> git merge feature-ai-enhancements
Already up to date.
PS C:\Users\nikhi\Projects\AI Assistant Coding\TestRepo>
```

Explanation

By managing the AI-generated code through Git branching, we isolate experimental fieatures firom the stable production codebase. This approach ensures that ifi an AI suggestion introduces unexpected bugs, the core application remains protected until the code is fiully reviewed and safiely merged.

Task 3 – AI fior Commit Message Generation

Task: * Use AI to generate concise, meaningfiul commit messages based on the changes made.

- Compare AI-generated messages with human-written ones.
- Discuss how AI can improve documentation and traceability in Git history.

Prompt

> "I just modified our Python project to add try-except blocks fior file I/O operations and added a new method to delete tasks. Generate 3 concise, meaningfiul Git commit messages fior these changes fiollowing conventional commit standards. Contrast them with a typical, lazy human commit message."

Code (Comparison Log)

```
--- Human-Written Commit Message ---
"fixed errors and added delete button"
"update todo.py again"

--- AI-Generated Commit Messages ---
Option 1: feat: add task deletion and implement robust file I/O error handling
Option 2: refactor: wrap file operations in try-except blocks and introduce delete feature
Option 3: feat(todo): support removing tasks and prevent crashes on file load failures
```

Output

```
# Applying the best AI-generated message
git commit -m "feat: add task deletion and implement robust file I/O error handling"
```

Explanation

Human-written commit messages often become vague and unhelpfiul when developers are rushing to save their work. The AI, however, consistently adheres to conventional commit standards, resulting in a much cleaner, highly searchable, and profiessional Git history that benefits the entire collaborative team.

Task 4 – Pair Programming Simulation

Task:

- Student A writes the initial version ofi a program.
- Student B, with AI assistance, suggests improvements or adds fieatures.

- Both students collaborate using Git branching and merging to integrate their work.

Prompt

> "Student A wrote a simple Python fiunction to calculate the average ofi a list (defi get_avg(nums): return sum(nums)/len(nums)). As Student B using AI, rewrite this fiunction to handle edge cases like empty lists and non-numeric values, and show how we would collaborate using Git."

Code

```python
# --- Student A's Initial Code (Main Branch) ---
def get_avg(nums):
    return sum(nums) / len(nums)

# --- Student B's AI-Assisted Improvement (Feature Branch) ---
def get_avg_improved(nums):
    """Calculates average, ignoring non-numeric types and handling empty lists."""
    if not nums:
        return 0.0

    # Filter out strings or None values
    valid_nums = [n for n in nums if isinstance(n, (int, float))]

    if not valid_nums:
        return 0.0

    return sum(valid_nums) / len(valid_nums)

# Testing the collaboration
print(f"Average: {get_avg_improved([10, 20, 'string', 30, None])}")
```

Output

```
Average: 20.0
```

Explanation

This exercise successfiully simulated real-world team dynamics where one developer establishes the base logic and another utilizes AI to fiortifiy the code against unpredictable edge cases. Git served as the essential bridge, allowing seamless parallel development and merging ofi both students' contributions without overwriting each other's work.